Purple Team Attack-to-Incident Walkthrough

1 - Introduction

In this project, I will be simulating 15 real attack techniques on my Windows 11 target using Atomic Red Team, analyse which ones Splunk detected and which it missed, write custom SPL alerts to close every gap, store those alerts as code in Github, and build a CI/CD pipeline that automatically validates every rule on every push.

1.1 - ATT&CK Techniques Simulated (REORDER)

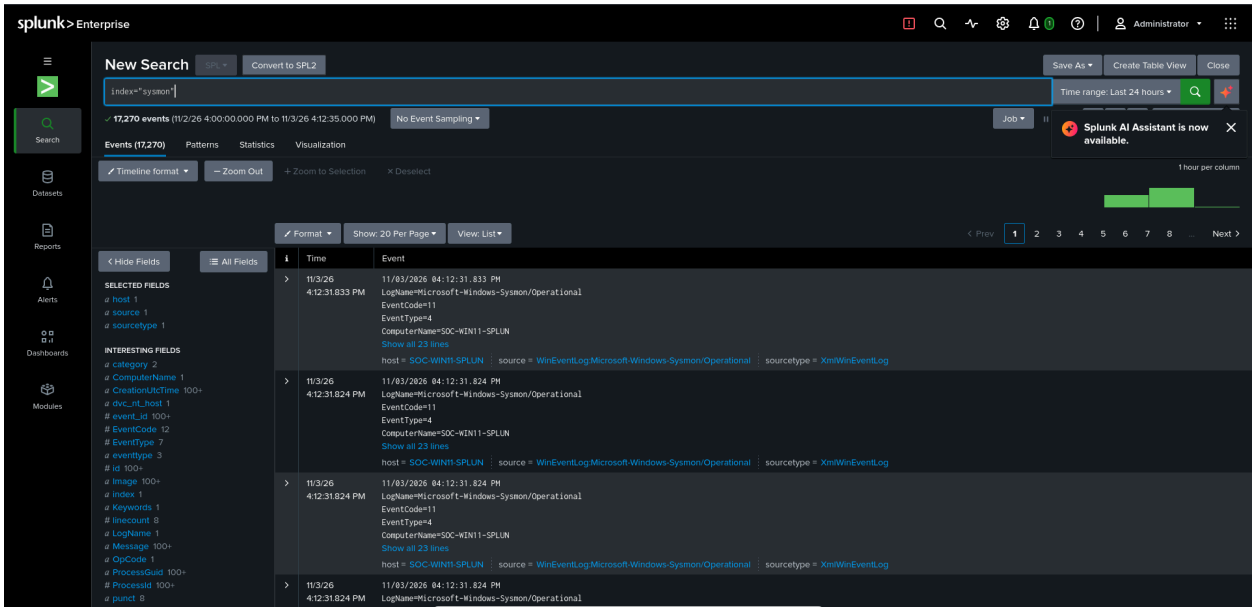
#	Technique	ID	Tactic
1	System Information Discovery	T1082	Discovery
2	Local Account Discovery	T1087.001	Discovery
3	Process Discovery	T1057	Discovery
4	Scheduled Task	T1053.005	Persistence
5	Registry Run Keys	T1547.001	Persistence
6	PowerShell Execution	T1059.001	Execution
7	Windows Command Shell	T1059.003	Execution
8	Modify Registry	T1112	Defense Evasion
9	Network Connection Discovery	T1049	Discovery
10	OS Credential Dumping: LSASS	T1003.001	Credential Access
11	Credential Dumping: SAM	T1003.002	Credential Access
12	Data from Local System	T1005	Collection
13	Ingress Tool Transfer	T1105	Command & Control
14	Masquerading	T1036	Defense Evasion
15	File and Directory Discovery	T1083	Discovery

1.2 - Lab Reference

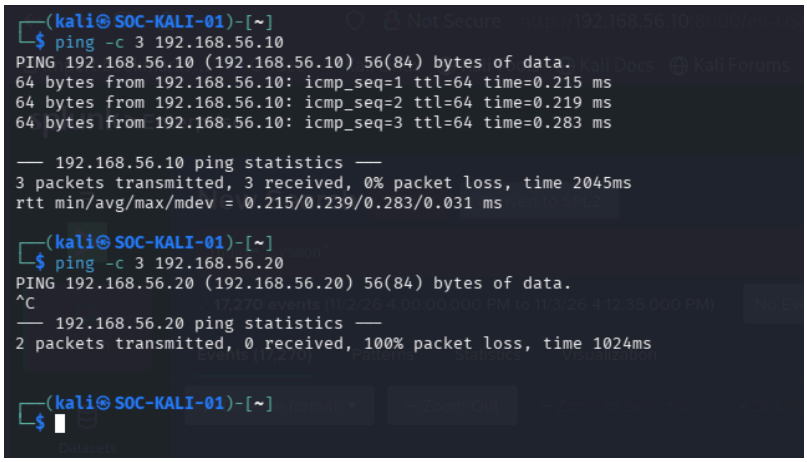
Machine	Role	Machine Name	IP
Ubuntu 26.04 LTS	SIEM - receives all attack telemetry	SOC-SPLUNK-01	192.168.56.10
Kali Linux	GitHub CLI and repo management	SOC-KALI-01	192.168.56.50
Windows 11 + Sysmon	Target Machine - Attack Red Team runs here	SOC-WIN11-01	192.168.56.20

1.3 - Project Prerequisites

- ☑ Snapshot of all 3 VMs (To ensure a clean state backup in case anything goes wrong)
- ☑ Confirmation that Splunk is receiving Sysmon logs from Windows (see image below)



- ☑ Confirming network connectivity between all machines (see image below)



  (Note, Windows does not allow ping requests and instantly drops them)
- ☑ Creation of a project folder on the Kali Machine
- ☑ Start time of project (Tuesday, November 3rd, 2026 4:14:13 PM)

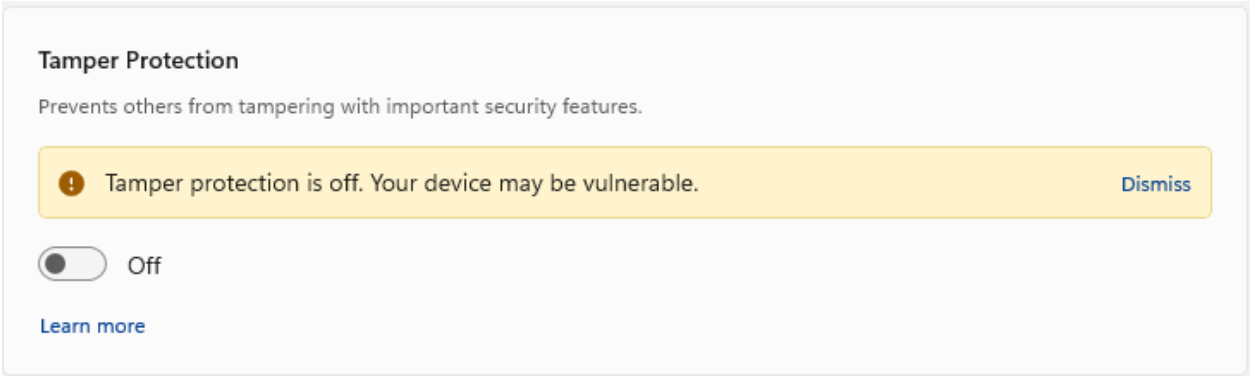
2 - Atomic Red Team Setup

Now that we made sure everything is working fine, lets start this project by first downloading the Atomic Red Team tool on windows. This is the tool that we will use to simulate all our attacks in order to be able to later on write our detection rules.

2.1 - Disabling Windows Defender and Updating Execution Policy

Before installing the tool, we first of all need to disable windows defender and add a few exclusions. We do this because Atomic Red Team is not a trusted windows tool and therefore when we download it, windows defender will automatically quarantine or delete it completely and therefore we need to disable it.

We will start by disabling tamper protection in the GUI settings of windows:



Then, we will disable windows defender and add exclusions using powershell:

```
PS C:\WINDOWS\system32> Set-MpPreference -DisableRealtimeMonitoring $true
>> Set-MpPreference -DisableBehaviorMonitoring $true
>> Set-MpPreference -DisableIOAVProtection $true
>> Add-MpPreference -ExclusionPath "C:\AtomicRedTeam"
>> Add-MpPreference -ExclusionPath "C:\Users\Public"
```

Then, we have to set the powershell execution policy in order to be able to bypass the execution policy and run scripts. By default, this policy on Windows is set to “Restricted” which blocks all .ps1 script files from running (including Atomic Red Team installer & all its test scripts).

```
PS C:\WINDOWS\system32> Set-ExecutionPolicy Bypass -Scope CurrentUser -Force
```

2.2 - Install Atomic Red Team on Windows

Now that we have done all of this, it’s finally time to install Atomic Red Team. We will download the powershell module via gallery and the full repository zip from github and extract it to the path the module expects:

```
None
# Force TLS 1.2 first
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12

# Install the dependency, then the module
Install-Module -Name powershell-yaml -Scope CurrentUser -Force
Install-Module -Name invoke-atomicredteam -Scope CurrentUser -Force

# Download the full repository zip
IWR -Uri "https://github.com/redcanaryco/atomic-red-team/archive/refs/heads/master.zip"
-OutFile "C:\AtomicRedTeam\art.zip" -UseBasicParsing

# Extract and move atomics to the path the module expects
Expand-Archive -Path "C:\AtomicRedTeam\art.zip" -DestinationPath "C:\AtomicRedTeam\" -Force
Move-Item "C:\AtomicRedTeam\atomic-red-team-master\atomics" "C:\AtomicRedTeam\atomics" -Force
```

After downloading, extracting, and moving the zip file, we will very both parts have installed correctly by running the following command:

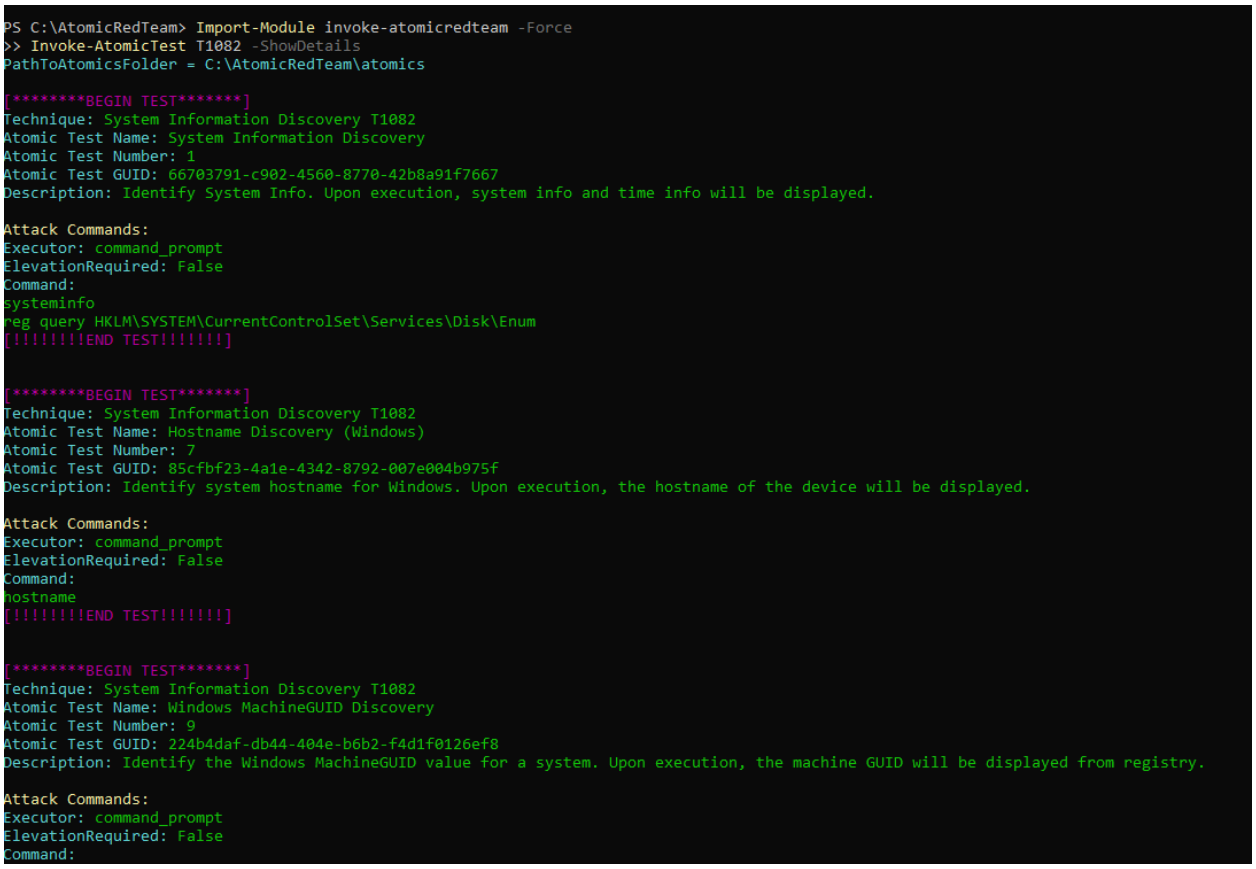
```
PS C:\AtomicRedTeam> Test-Path "C:\AtomicRedTeam\atomics"
>> Get-ChildItem "C:\AtomicRedTeam\atomics" | Select-Object -First 5
True


    Directory: C:\AtomicRedTeam\atomics


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        11/3/2026   5:47 PM                Indexes
d-----        11/3/2026   5:47 PM                T1001.002
d-----        11/3/2026   5:47 PM                T1003
d-----        11/3/2026   5:47 PM                T1003.001
d-----        11/3/2026   5:47 PM                T1003.002
```

As evident from the screenshot, the returned output is “True, followed by a list of technique folders which indicates the powershell modules have been installed and the atomics library has landed at “C:\AtomicRedTeam\atomics\”.

We will then import the module and verify using an atomic test that the module is imported correctly:



The output is a long list of all the available T1082 tests where each block shows the test name, number, description, and the exact commands it runs.

Finally, we will add the module to a powershell profile so that it auto-loads the module every time we open powershell.

```
None
Add-Content $PROFILE 'Import-Module invoke-atomicredteam -Force'
```

3 - Run The Attack Techniques

Now that everything is ready and good to go, it's time to start running the attacks and checking splunk to make sure the attacks are being logged. Before doing so, we will check the system date and time to know exactly when the attacks started (“[System.DateTime]::Now”):

Tuesday, November 3, 2026 4:21:30 PM

For every technique we run, we will follow the following workflow:

1. Note the UTC time: [System.DateTime]::Now
2. Run the Technique
3. Check Splunk for a log
4. Note YES or NO in our detection gap log
5. Run a cleanup command to remove effects of technique

3.1 - T1082 — System Information Discovery

The first technique is the System Information Discover (T1082) which simply runs the “systeminfo” command in powershell in order to gain insights about the target system, usually executed by attackers in the reconnaissance phase. We will only be running tests 1 and 7 in this test.

1.

```
PS C:\WINDOWS\system32> [System.DateTime]::Now

Tuesday, November 3, 2026 6:33:29 PM
```

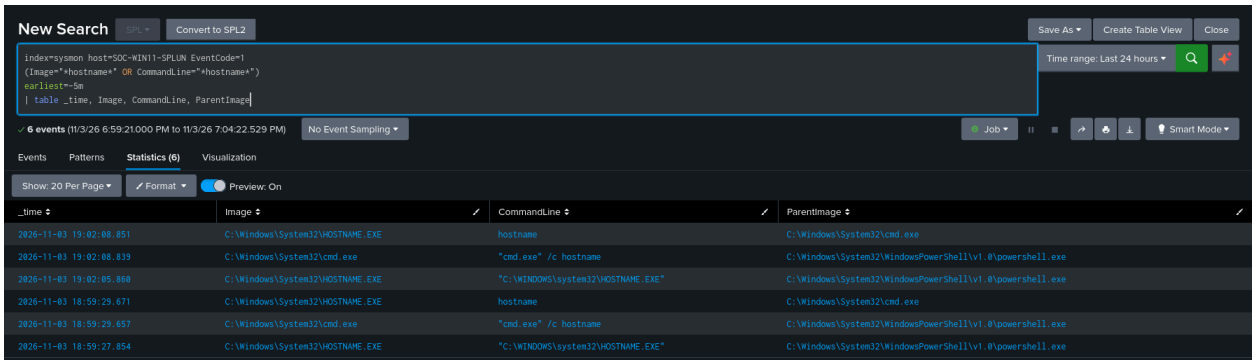
2.

```
PS C:\WINDOWS\system32> Invoke-AtomicTest T1082 -TestNumbers 1,7
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

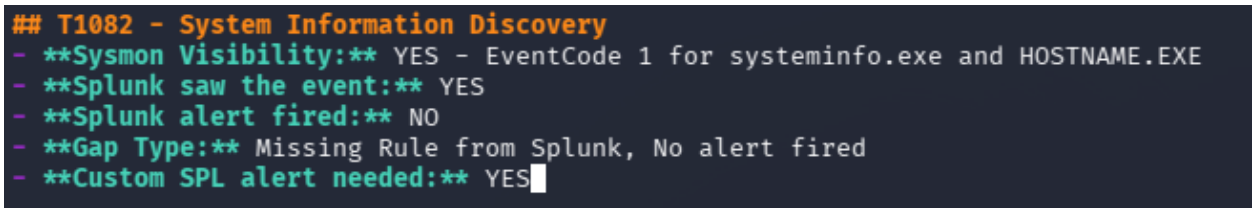
Executing test: T1082-1 System Information Discovery
Host Name:                 SOC-WIN11-SPLUN
OS Name:                   Microsoft Windows 11 Pro
OS Version:                10.0.26200 N/A Build 26200
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:             Multiprocessor Free
Registered Organization:   N/A
Product ID:                00330-80000-00000-AA693
Registered Owner:          N/A
Original Install Date:     10/25/2026, 2:37:18 PM
System Boot Time:          11/3/2026, 1:51:58 PM
System Manufacturer:       innotek GmbH
System Model:              VirtualBox
System Type:               x64-based PC
Processor(s):              1 Processor(s) Installed.
                           [01]: Intel64 Family 6 Model 198 Stepping 2 GenuineIntel ~3686 Mhz
BIOS Version:              innotek GmbH VirtualBox, 12/1/2006
Windows Directory:         C:\WINDOWS
System Directory:          C:\WINDOWS\system32
Boot Device:               \Device\HarddiskVolume2
System Locale:             en-us;English (United States)
Input Locale:              en-us;English (United States)
Time Zone:                 (UTC+02:00) Cairo
Total Physical Memory:     4,077 MB
Available Physical Memory: 1,815 MB
Virtual Memory: Max Size:  5,485 MB
Virtual Memory: Available: 1,751 MB
Virtual Memory: In Use:    3,734 MB
Page File Location(s):     C:\pagefile.sys
Domain:                    WORKGROUP
Logon Server:              \\SOC-WIN11-SPLUN
Hotfix(s):                 4 Hotfix(s) Installed.
                           [01]: KB5127690
                           [02]: KB5054156
                           [03]: KB5079473
                           [04]: KB5127647
Network Card(s):           2 NIC(s) Installed.
                           [01]: Intel(R) PRO/1000 MT Desktop Adapter
                                 Connection Name: Ethernet
                                 DHCP Enabled:    Yes
                                 DHCP Server:     10.0.2.2
                                 IP address(es)
                                 [01]: 10.0.2.15
                                 [02]: fe80::a509:8173:2497:31e5
                                 [03]: fd17:625c:f037:2:f592:bb8:e206:428f
                                 [04]: fd17:625c:f037:2:c967:53d9:b19d:f45b
                           [02]: Intel(R) PRO/1000 MT Desktop Adapter
                                 Connection Name: Ethernet 2
                                 DHCP Enabled:    No
                                 IP address(es)
                                 [01]: 192.168.56.20
                                 [02]: fe80::33aa:d7b2:783:3b5f
Virtualization-based security: Status: Not enabled
                               App Control for Business policy: Enforced
                               App Control for Business user mode policy: Off
                               Security Features Enabled:
Hyper-V Requirements:          A hypervisor has been detected. Features required for Hyper-V will not be displayed.
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Disk\Enum
    0    REG_SZ    SCSI\Disk&Ven_VBOX&Prod_HARDDISK\4&2617aeae&0&000000
    Count    REG_DWORD    0x1
    NextInstance    REG_DWORD    0x1
Exit code: 0
Done executing test: T1082-1 System Information Discovery
Executing test: T1082-7 Hostname Discovery (Windows)
SOC-WIN11-SPLUN
Exit code: 0
Done executing test: T1082-7 Hostname Discovery (Windows)
```

3.

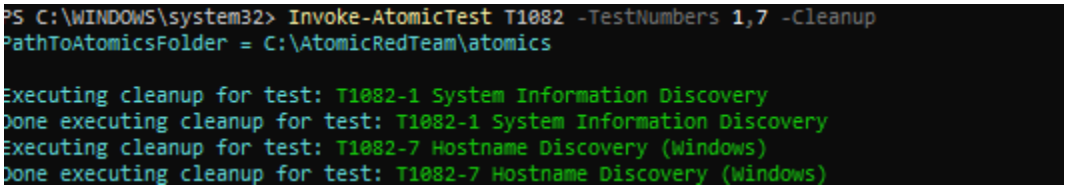




4.



5.



We have successfully executed our first test technique using the tool where we ran the test technique, checked splunk with search queries to see if Sysmon logged the activity, checked whether an alert was triggered in splunk alerts, noted everything in our detection gap log, and finally cleaned up any traces left by the attack.

Now, the next step is to do the same exact workflow for all the other test techniques but in order to speed up the process, I will not be showing every step for every technique, only mentioning the technique and the commands used. A finished version of the detection gap log can be found in the GitHub repository indicating all the detection gaps that were found in this lab.

3.2 - T1087.001 - Local Account Discovery

What it does: Runs net user and net localgroup administrators to list all local accounts and members of the Administrators group. Attackers use this to identify high-value accounts to target for credential theft or privilege escalation.

Command:

```
None
Invoke-AtomicTest T1087.001 -TestNumbers 8,9,10,11
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(CommandLine="*net user*" OR CommandLine="*net localgroup*" OR
 CommandLine="*Get-LocalUser*" OR CommandLine="*Get-LocalGroupMember*" OR
 Image="*net1.exe*")
earliest=-5m
| table _time, Image, EventCode, CommandLine, ParentImage
```

3.3 - T1057 - Process Discovery

What it does: Runs tasklist to enumerate all running processes on the system. Attackers use this to identify security tools (AV, EDR), find processes to inject into, or confirm their malware is running.

Command:

```
None
Invoke-AtomicTest T1057 -TestNumbers 2,3
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(Image="*tasklist*" OR CommandLine="*Get-Process*")
earliest=-5m
| table _time, Image, EventCode, CommandLine
```

3.4 - T1053.005 - Scheduled Task

What it does: Creates a Windows scheduled task using schtasks /create so that a specified program runs automatically at login or on a schedule. Attackers use scheduled tasks as one of the most common persistence mechanisms — the task survives reboots and runs without user interaction.

Command:

```
None
Invoke-AtomicTest T1053.005 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*schtasks*" CommandLine="*/create*"
earliest=-5m
| table _time, Image, EventCode, CommandLine
```

3.5 - T1547.001 - Registry Run Keys

What it does: Adds a value to HKCU\Software\Microsoft\Windows\CurrentVersion\Run pointing to an executable. Windows runs anything listed in this key automatically when the user logs in. One of the most widely abused persistence locations in real-world malware. Sysmon Event ID 13 (RegistryEvent value set) targeting Run keys is one of the most valuable detection rules you can write.

Command:

```
None
Invoke-AtomicTest T1547.001 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=13
TargetObject="*CurrentVersion\\Run*"
earliest=-5m
| table _time, Image, EventCode, TargetObject, Details
```

3.6 - T1059.001 - PowerShell Execution

What it does: Executes a Base64-encoded PowerShell command using the -EncodedCommand flag. Attackers encode their scripts to bypass detection rules that look for plaintext keywords and to avoid string-based scanning. The actual command is invisible in the process command line; it looks like a random string of characters.

Command:

```
None
Invoke-AtomicTest T1059.001 -TestNumbers 10,17
```

Splunk Query:

```
None
## Detects test 17
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*powershell*"
(CommandLine="*-enc*" OR CommandLine="*EncodedCommand*" OR
 CommandLine="*IEX*" OR CommandLine="*FromBase64String*" OR
 CommandLine="* -e *")
earliest=-5m
| table _time, Image, CommandLine, ParentImage
-------------------------------------------------------------------------------
## Detects Test 10
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*reg.exe*"
earliest=-5m
| table _time, Image, CommandLine, ParentImage
```

3.7 - T1059.003 - Windows Command Shell

What it does: Spawns cmd.exe from an unusual parent process. Legitimate cmd.exe sessions are almost always launched from explorer.exe (user double-clicking) or another cmd. When cmd.exe is spawned by PowerShell, a script, or a document, it is a strong indicator of malicious activity.

Command:

```
None
Invoke-AtomicTest T1059.003 -TestNumbers 2,3
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*cmd.exe" ParentImage!="*explorer.exe"
earliest=-5m
| table _time, Image, EventCode, CommandLine, ParentImage
```

3.8 - T1112 - Modify Registry

What it does: Uses reg.exe to add or modify a registry value. Attackers modify the registry for persistence (Run keys), defence evasion (disabling security tools via policy keys), and configuration changes.

Command:

```
None
Invoke-AtomicTest T1112 -TestNumbers 1,40
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=13
Image="*reg.exe*"
earliest=-5m
| table _time, Image, TargetObject, Details
```

3.9 - T1049 - Network Connection Discovery

What it does: Runs netstat to enumerate all active network connections and listening ports. Attackers use this to confirm their C2 connection is active, find other reachable hosts, and understand what network services are exposed.

Command:

```
None
Invoke-AtomicTest T1049 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(Image="*netstat.exe*" OR CommandLine="*net use*" OR CommandLine="*net share*")
earliest=-5m | table _time, Image, CommandLine
```

3.10 - T1003.001 - LSASS Memory Dump

What it does: Uses the built-in Windows comsvcs.dll to create a memory dump of the LSASS process (Local Security Authority Subsystem Service). LSASS holds the credentials of logged-in users - including plaintext passwords and NTLM hashes.

Command:

```
None
Invoke-AtomicTest T1003.001 -TestNumbers 2
```

Splunk Query:

```
None
## Primary detection — comsvcs.dll MiniDump command line (catches Test 2):
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
CommandLine="*comsvcs*" CommandLine="*MiniDump*"
earliest=-5m
| table _time, Image, CommandLine

## Secondary check — LSASS process access (may require Sysmon ProcessAccess config
tuning):
index=sysmon host=SOC-WIN11-SPLUN EventCode=10
TargetImage="*lsass*"
earliest=-5m
| table _time, SourceImage, TargetImage, GrantedAccess
```

3.11 - T1003.002 - SAM Database Dump

What it does: Uses reg save to export the SAM (Security Account Manager) database to a file. The SAM stores hashed passwords for all local accounts. Combined with the SYSTEM hive (which contains the boot key needed to decrypt it), these hashes can be cracked offline or used directly in Pass-the-Hash attacks.

Command:

```
None
Invoke-AtomicTest T1003.002 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
CommandLine="*reg save*" (CommandLine="*SAM*" OR CommandLine="*SYSTEM*" OR
CommandLine="*SECURITY*") earliest=-5m | table _time, Image, CommandLine
```

3.12 - T1005 - Data from Local System

What it does: Runs dir /s to recursively enumerate all files and directories. Attackers use this to locate sensitive files - credentials stored in config files, documents, SSH keys - before staging them for exfiltration.

Command:

```
None
Invoke-AtomicTest T1005 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*powershell*"
"Compress-Archive"
earliest=-5m
| table _time, Image, CommandLine | sort -_time
```

3.13 - T1105 - Ingress Tool Transfer

Uses certutil.exe with the -urlcache -split -f flags to download a file from the internet. certutil.exe is a built-in Windows certificate utility that attackers abuse as a Living-off-the-Land Binary (LOLBin) to download payloads without spawning obviously suspicious tools. It is widely used by real malware families because it is signed by Microsoft, whitelisted by many security tools, and present on every Windows machine.

Command:

```
None
Invoke-AtomicTest T1105 -TestNumbers 7
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN
(EventCode=3 Image="*certutil*") OR
(EventCode=1 Image="*certutil*" CommandLine="*urlcache*")
earliest=-5m
| table _time, EventCode, Image, CommandLine, DestinationIp
```

3.14 - T1036 - Masquerading

Copies a legitimate Windows binary and places it in a non-standard location with a similar-sounding name. For example, copying cmd.exe to C:\Users\Public\svch0st.exe. Attackers do this to bypass detection rules that look only at process names - the process name looks legitimate but is running from a suspicious path.

Command:

```
None
Invoke-AtomicTest T1036 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*cmd.exe"
NOT (Image="C:\\Windows\\System32\\cmd.exe" OR
Image="C:\\Windows\\SysWOW64\\cmd.exe")
earliest=-5m
| table _time, Image, EventCode, CommandLine, ParentImage
```

3.15 - T1083 - File and Directory Discovery

What it does: Uses dir /s to enumerate the file and directory structure of the system. Similar to T1005 (Data from Local System) but focused specifically on mapping the directory tree rather than reading file contents. Attackers often run both T1082 (system info) and T1083 (file discovery) together during the initial reconnaissance phase.

Command:

```
None
Invoke-AtomicTest T1083 -TestNumbers 1
```

Splunk Query:

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(CommandLine="*dir /s*" OR CommandLine="*tree /F*"
OR (CommandLine="*Get-ChildItem*" CommandLine="*recurse*")
OR CommandLine="*gci -recurse*" OR CommandLine="*ls -recurse*")
earliest=-5m
| table _time, Image, CommandLine, ParentImage
```

Now that we have run all 15 attack tests from the table in (1.1), it is time to actually start the detection-as-code pipeline where we will be Writing a custom Splunk alert for every detection gap, saving each as a .spl file, initialising the GitHub repo, and building the GitHub Actions CI/CD validation workflow.

4 - Write SPL Alerts and Set Up GitHub

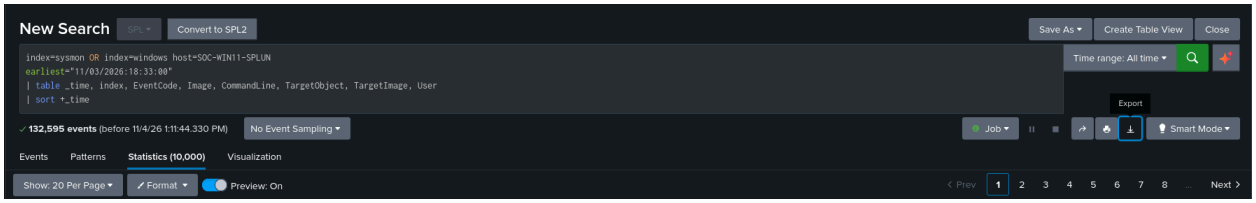
What we will now be doing is writing custom Splunk alerts for every detection gap, saving them as individual .spl files, initialising the Github repo and building the GitHub Actions CI/CD validation workflow.

4.1 - Write and Save the SPL Alerts

For each alert, we will follow the following workflow:

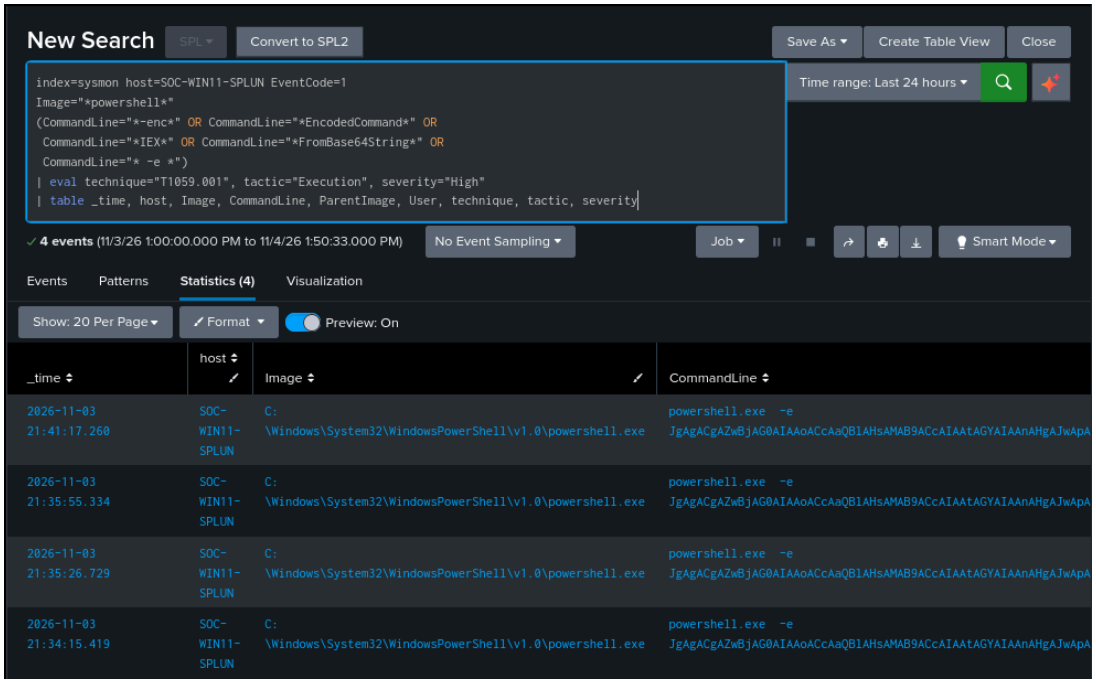
1. Write the SPL into Splunk Search & Reporting
2. Confirm it returns at least one row from our Atomic Red Team data
3. Click Save As -> Alert with the name, severity, real-time trigger, and results > 0 specified

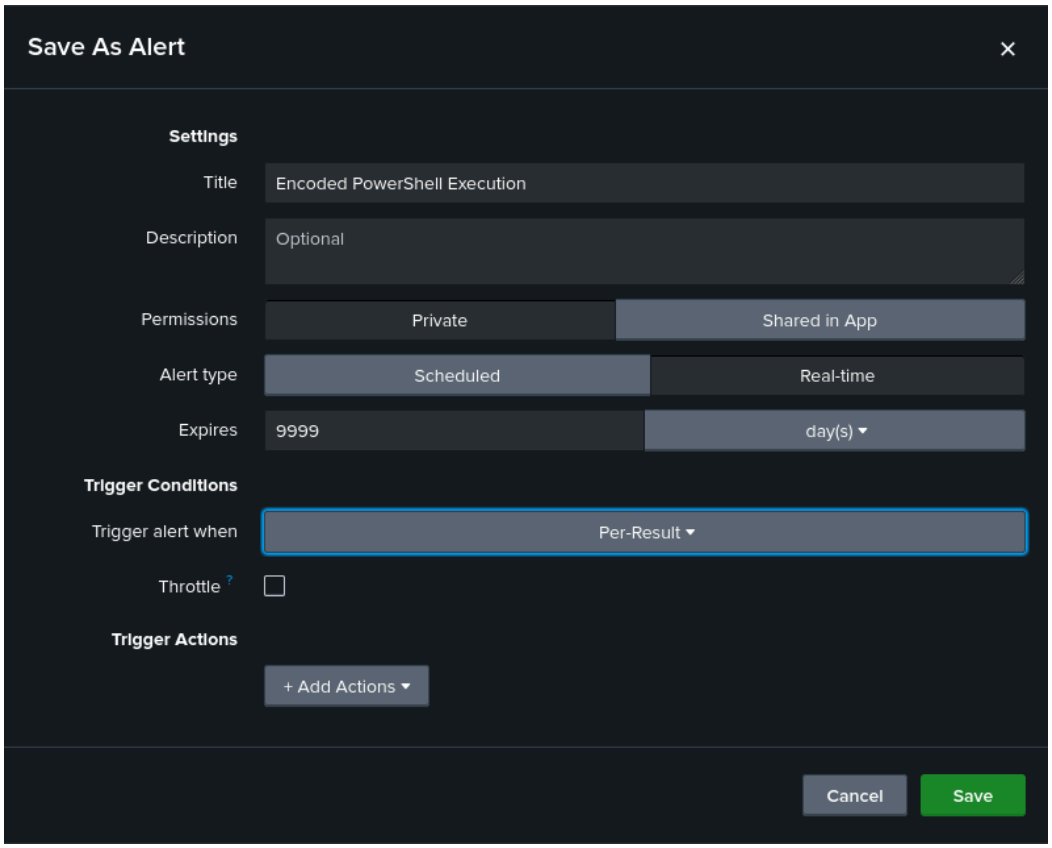
Before doing this, let's first extract all the alerts that were generated since the beginning of our test phase as a way to keep track of all the logs generated by the tests and all the noise around them:



Now, it's time to start writing the SPL queries:

4.1.1 - Alert 1a: Encoded PowerShell Execution (T1059.001, Execution, Severity: High)





```
┌──(kali㉿SOC-KALI-01)-[~/projects/project1/detections]
└─$ cat > ~/projects/project1/detections/T1059.001_encoded_powershell.spl << 'EOF'
# ATT&CK: T1059.001 - Encoded PowerShell Execution
# Tactic: Execution
# Severity: High
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*powershell*"
(CommandLine="*-enc*" OR CommandLine="*EncodedCommand*" OR
 CommandLine="*IEX*" OR CommandLine="*FromBase64String*" OR
 CommandLine="* -e *")
| eval technique="T1059.001", tactic="Execution", severity="High"
| table _time, host, Image, CommandLine, ParentImage, User, technique, tactic, severity
EOF
```

For the sake of simplicity, I will not be taking screenshots for every alert, I will only be writing the alert that will later on be added to the .spl file and the file can be found in the GitHub repository file known as “detections”.

4.1.2 - Alert 2: cmd.exe Spawned by Unusual Parent (T1059.003, Execution, Medium)

None

```
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*\\cmd.exe"
CommandLine="*LOCALAPPDATA*" OR CommandLine="*~-3,1*"
| eval technique="T1059.003", tactic="Execution", severity="Medium"
| table _time, host, Image, EventCode, CommandLine, ParentImage,
User, technique, tactic, severity
```

4.1.3 - Alert 3: Registry Modified via reg.exe (T1112, Defense Evasion, Medium)

None

```
index=sysmon host=SOC-WIN11-SPLUN EventCode=13
Image="*reg.exe*"
| eval technique="T1112", tactic="Defense Evasion",
severity="Medium"
| table _time, host, Image, TargetObject, Details, User, technique,
tactic, severity
```

4.1.4 - Alert 4: System Information Discovery (T1082, Discovery, Low)

None

```
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(Image="*systeminfo*" OR Image="*hostname.exe*" OR
CommandLine="*systeminfo*")
NOT (CommandLine="*C:\*")
| eval technique="T1082", tactic="Discovery", severity="Low"
```

```
| table _time, host, Image, CommandLine, ParentImage, User,
technique, tactic, severity
```

4.1.5 - Alert 5: Local Account Enumeration (T1087.001, Discovery, Low)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(CommandLine="*net user*" OR CommandLine="*net localgroup*" OR
 CommandLine="*Get-LocalUser*" OR
CommandLine="*Get-LocalGroupMember*" OR
 Image="*net1.exe*")
| eval technique="T1087.001", tactic="Discovery", severity="Low"
| table _time, host, Image, EventCode, CommandLine, ParentImage,
User, technique, tactic, severity
```

4.1.6 - Alert 6: Process Discovery (T1057, Discovery, Low)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(Image="*tasklist*" OR CommandLine="*Get-Process*")
| eval technique="T1057", tactic="Discovery", severity="Low"
| table _time, host, Image, EventCode, CommandLine, User,
technique, tactic, severity
```

4.1.7 - Alert 7: LSASS Dump via comsvcs.dll (T1003.001, Credential Access, Critical)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
CommandLine="*comsvcs*" CommandLine="*MiniDump*"
| eval technique="T1003.001", tactic="Credential Access",
severity="Critical"
| table _time, host, Image, CommandLine, User, technique, tactic,
severity
```

4.1.8 - Alert 8: SAM Database Dump (T1003.002, Credential Access, Critical)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
CommandLine="*reg save*"
(CommandLine="*SAM*" OR CommandLine="*SYSTEM*" OR
CommandLine="*SECURITY*")
| eval technique="T1003.002", tactic="Credential Access",
severity="Critical"
| table _time, host, Image, CommandLine, User, technique, tactic,
severity
```

4.1.9 - Alert 9: Data Collection via Compress-Archive (T1005, Collection, Low)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*powershell*"
"Compress-Archive"
| eval technique="T1005", tactic="Collection", severity="Low"
| table _time, host, Image, CommandLine, User, technique, tactic,
severity
```

4.1.10 - Alert 10: Network Connection Discovery (T1049, Discovery, Low)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(Image="*netstat.exe*" OR CommandLine="*net use*" OR
CommandLine="*net share*")
| eval technique="T1049", tactic="Discovery", severity="Low"
| table _time, host, Image, CommandLine, User, technique, tactic,
severity
```

4.1.11 - Alert 11: LOLBin File Download via certutil (T1105, Command and Control, High)

```
None
index=sysmon host=SOC-WIN11-SPLUN
(EventCode=3 Image="*certutil*") OR
(EventCode=1 Image="*certutil*" CommandLine="*urlcache*")
| eval technique="T1105", tactic="Command and Control",
severity="High"
| table _time, host, EventCode, Image, CommandLine, DestinationIp,
User, technique, tactic, severity
```

4.1.12 - Alert 12: System Binary Masquerading (T1036, Defense Evasion, High)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*cmd.exe"
NOT (Image="C:\\Windows\\System32\\cmd.exe" OR
Image="C:\\Windows\\SysWOW64\\cmd.exe")
| eval technique="T1036", tactic="Defense Evasion", severity="High"
| table _time, host, Image, EventCode, CommandLine, ParentImage,
User, technique, tactic, severity
```

4.1.13 - Alert 13: Scheduled Task Created (T1053.005, Persistence, High)

```
None
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
Image="*schtasks*" CommandLine="*/create*"
| eval technique="T1053.005", tactic="Persistence", severity="High"
| table _time, host, Image, EventCode, CommandLine, User,
technique, tactic, severity
```

4.1.14 - Alert 14: Registry Run Key Modified (T1547.001, Persistence, High)

None

```
index=sysmon host=SOC-WIN11-SPLUN EventCode=13
TargetObject="*CurrentVersion\\Run*"
| eval technique="T1547.001", tactic="Persistence", severity="High"
| table _time, host, Image, EventCode, TargetObject, Details, User,
technique, tactic, severity
```

4.1.15 - Alert 15: File and Directory Discovery (T1083, Discovery, Low)

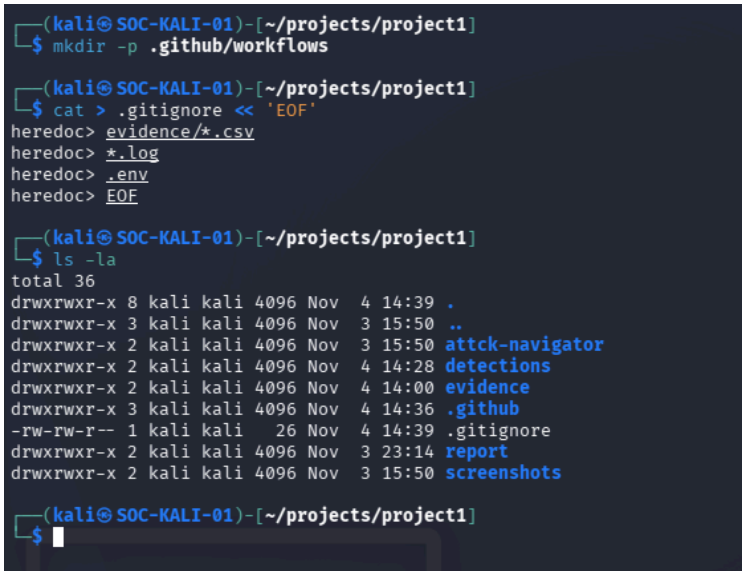
None

```
index=sysmon host=SOC-WIN11-SPLUN EventCode=1
(CommandLine="*dir /s*" OR CommandLine="*tree /F*"
OR (CommandLine="*Get-ChildItem*" CommandLine="*recurse*")
OR CommandLine="*gci -recurse*" OR CommandLine="*ls -recurse*")
| eval technique="T1083", tactic="Discovery", severity="Low"
| table _time, host, Image, CommandLine, ParentImage, User,
technique, tactic, severity
```

4.2 - Set Up GitHub and CI/CD Pipeline

Now that we have written all the alerts in Splunk and added them as .spl files in our github repository, it is time to set up our github workflow and CI/CD pipeline.

First, we will start by creating our github repository structure:



Then, we will write the GitHub actions validation workflow that will automatically validate every spl rule that we upload to the github repository:

```
┌──(kali㉿SOC-KALI-01)-[~/projects/project1]
└─$ cat > .github/workflows/validate-detections.yml << 'EOF'
name: Validate SPL Detections

on:
  push:
    paths:
      - 'detections/**'
  pull_request:
    paths:
      - 'detections/**'

jobs:
  validate:
    runs-on: ubuntu-latest
    name: Validate SPL detection files

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Check all .spl files are non-empty
        run: |
          FAILED=0
          for file in detections/*.spl; do
            if [ ! -s "$file" ]; then
              echo "FAIL: $file is empty"
              FAILED=1
            else
              echo "PASS: $file"
            fi
          done
          exit $FAILED

      - name: Check each .spl file has ATT&CK mapping comment
        run: |
          FAILED=0
          for file in detections/*.spl; do
            if ! grep -q "# ATT&CK:" "$file"; then
              echo "FAIL: $file missing ATT&CK comment"
              FAILED=1
            else
              echo "PASS: $file - $(grep '# ATT&CK:' $file | head -1)"
            fi
          done
          exit $FAILED

      - name: Check each .spl file has Severity label
        run: |
          FAILED=0
          for file in detections/*.spl; do
            if ! grep -q "# Severity:" "$file"; then
              echo "FAIL: $file missing Severity label"
              FAILED=1
            else
              echo "PASS: $file - $(grep '# Severity:' $file | head -1)"
            fi
          done
          exit $FAILED

      - name: Print detection summary
        run: |
          COUNT=$(ls detections/*.spl 2>/dev/null | wc -l)
          echo "Total SPL detections: $COUNT"
          echo "All validation checks passed."
EOF
```

This is the main part of this project (CI/CD Pipeline) which goes to every spl file and checks it for the following criteria:

1. .spl file is not empty
2. .spl file has ATT&CK mapping comment
3. .spl file has Severity label

Then it prints the detection summary, if a file fails any of the checks mentioned above then it is rejected by the github repository and automatically removed from the push request and an alert is returned to the user with a message indicating which check the file failed.